

SwasthyaSetu

A Rural Healthcare Continuity Platform

SIH 2026 Prototype — Technical Overview

1. Overview

SwasthyaSetu addresses a specific, concrete gap in rural healthcare delivery: a patient seen by a field worker (ANM/ASHA), then referred to a doctor at a different facility, then perhaps a specialist, has no single continuous record. Each encounter is siloed, so the patient re-explains their history every time, and no one downstream has visibility into what already happened upstream.

The platform gives every patient one shared, consent-controlled health record that follows them across facilities, workers, and time — accessible via an opaque QR health ID, built as a modular monolith (one mobile codebase for every role, one backend with separated domain modules) rather than microservices, to stay within a zero-cost, rapidly-buildable stack suitable for a hackathon prototype.

The system was built phase by phase against a fixed architecture document, with each phase gated by an explicit exit test before moving to the next — sequenced so that identity and access control (Phases 1–3) were solid before referrals (Phase 4), offline support (Phase 5), realtime features (Phase 6), AI assistance (Phase 7), accessibility (Phase 8), and operational oversight (Phase 9) were layered on top.

2. Roles and applications

Four roles use the system, split across two client applications and one shared backend:

Role	Application	Purpose
Patient	Mobile app	View own QR health ID and timeline; grant/revoke consent; track referral status
ANM / ASHA (field worker)	Mobile app	Register patients; record visits; create referrals — works fully offline
Doctor	Mobile app	All ANM/ASHA actions, plus accept/complete referrals, teleconsultation, and AI summary review + approval
District Admin	Admin console (separate web app)	Read-only operational oversight: bottlenecks, facility load, sync health, demo-data reset

All three deployable applications — the Expo/React Native mobile app, the React/Vite admin console, and the Express API — share one TypeScript type package, so a field the backend returns and a field a client expects to receive can never silently drift apart.

3. System architecture

Both client applications talk exclusively to one Express API; neither ever reaches the database or an AI provider directly. This is a deliberate choke point: every access-control rule, every audit log entry, and every piece of business logic exists in exactly one place, never duplicated between a client and the server, and never duplicated into database-level policies.

Layer	Technology	Responsibility
Mobile client	Expo (React Native), Expo Router	Patient / ANM-ASHA / doctor UI, offline outbox, WebRTC teleconsultation
Admin client	React + Vite	District dashboard, aggregated metrics, demo-data reset
Backend	Node.js, Express, Socket.IO	REST API, access control, audit logging, realtime notifications

Layer	Technology	Responsibility
Database	Supabase (Postgres + Auth)	Source of truth for all data; row-level auth for a user's own profile row only
AI adapters	Groq, Google Gemini	Optional model-authored summary drafts, validated against a shared JSON Schema

Realtime updates (Socket.IO) are treated strictly as a notification, never as a data source: every screen re-fetches the authoritative REST data on receiving an event, so a missed or duplicated socket message can never leave the UI showing something the database disagrees with.

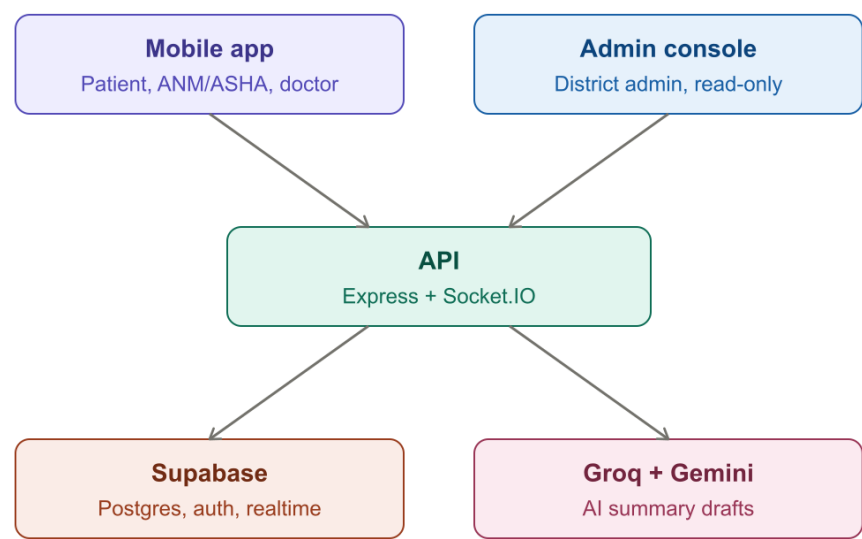


Figure 1. Both clients talk only to the API; the API is the only thing that reaches Supabase or the AI providers.

4. Phase-by-phase build

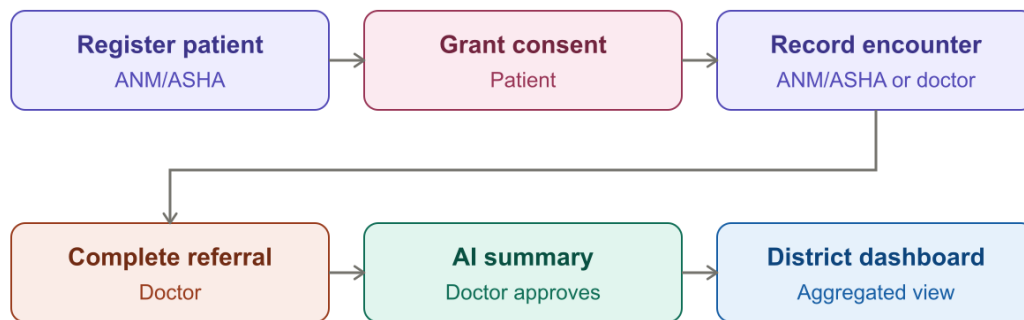


Figure 2. The core patient journey, colored by who acts at each step — purple for field staff, pink for the patient, coral for cross-facility referral, teal for the AI-assisted step, blue for the read-only admin view.

Each phase below was gated by its own exit test before the next one began.

Phase 0 — Foundation

Build: Monorepo scaffold (npm workspaces), Expo mobile app, Express API, Supabase project wiring.

Guardrail: No real patient data anywhere in this environment — synthetic data only.

Exit test: Mobile app reports a live “connected” status from the API and database.

Phase 1 — Authentication & role-based access

Build: Supabase Auth, a profiles table extending auth.users with role and facility, Express middleware (verifyAuth, requireRole).

Guardrail: Role is enforced server-side, never inferred from what the client UI happens to show.

Exit test: All four roles see different navigation, and an admin-only endpoint returns 200 only for district_admin.

Phase 2 — Patient profile & QR digital health ID

Build: Patient record with an opaque health ID (SS-XXXXXXXXXX) that carries no medical information; QR generation and camera-based scanning.

Guardrail: The QR code itself is inert — all authorization happens after it resolves, server-side.

Exit test: A field worker registers a patient, generates a QR, and a second device scanning it reaches the same authorized record.

Phase 3 — Consent, encounter timeline, and audit

Build: Additive/revocable consent rows (not a boolean flag); append-only encounters; a single canAccessPatient rule (home facility match or active consent) reused everywhere.

Guardrail: Access control lives in exactly one place — never duplicated into database policies.

Exit test: A doctor at a different facility is blocked until consent is granted, then gains access, then loses it immediately on revocation — all logged.

Phase 4 — Referral continuity

Build: A referral state machine (pending → accepted → completed, or cancelled/reassigned), with realtime status updates over Socket.IO.

Guardrail: Referral access is granted to the two facilities the referral is between — separate from general patient consent.

Exit test: A referral created by one facility is accepted and completed by another, updating both the referral and the patient's timeline.

Phase 5 — Offline-first field app & sync

Build: An on-device outbox (SQLite on native, browser storage on web) that queues visits and referrals created without connectivity, syncing automatically on reconnect.

Guardrail: No blind last-write-wins on clinical facts — encounters stay append-only.

Exit test: A retried sync request, carrying an Idempotency-Key, lands on the same database row instead of creating a duplicate.

Phase 6 — Low-bandwidth teleconsultation

Build: WebRTC audio/video calls signaled through a Socket.IO room scoped to an accepted referral; the server relays signaling data without inspecting or storing it.

Guardrail: Public STUN only, no paid TURN infrastructure — works on direct/LAN paths, an intentional free-tier limitation.

Exit test: Two devices exchange a real signaling handshake and reach a connected call state; ending the call writes an encounter with duration.

Phase 7 — AI summarization & assisted triage

Build: A deterministic template is always available as the guaranteed baseline; Groq is tried first, Gemini second, each validated against an identical JSON Schema before being trusted.

Guardrail: AI is assistance, not autonomous diagnosis — a doctor must approve before a patient ever sees a summary.

Exit test: A generated draft is edited and approved by a doctor, with the model name and adapter version recorded permanently.

Phase 8 — Multilingual voice & accessibility

Build: English/Hindi/Telugu via expo-localization and i18n-js, defaulting to the device's own language; icon-led actions; large touch targets; step-numbered guided prompts; speech input on the web build via the browser's native SpeechRecognition API.

Guardrail: Voice complements the interface — typing and tapping always remain fully functional everywhere.

Exit test: Switching language re-translates every screen, including native header titles, with no navigation reset.

Phase 9 — District dashboard & hardening

Build: A dedicated React/Vite admin console (not a mobile screen) showing referral bottlenecks split into accept-latency and complete-latency, facility load, sync health, and audit activity, plus a guarded demo-data reset.

Guardrail: The dashboard is read-optimized and aggregated only — no patient name or clinical note ever crosses this endpoint.

Exit test: An admin inspects real bottleneck and facility-load numbers with no clinical detail, and can reset all test data back to a clean slate.

5. The AI summary flow, in detail

Generating a summary escalates through an ordered chain, never blocking on a single provider's availability:

- A deterministic template is built first, directly from the patient's recorded encounters — this text is both the input the models are asked to rewrite, and the guaranteed fallback if nothing else succeeds.
- Groq is tried first. Its raw JSON reply is validated against a shared schema (via ajv) — a model's own "structured output" mode is a hint to the model, not a guarantee, so nothing is trusted until it passes real validation.
- If Groq is unset, fails, or returns something that fails validation, Gemini is tried next, through the identical schema.
- If both fail, the plain template text is saved as the draft with no model attached — the feature never has a hard failure mode.

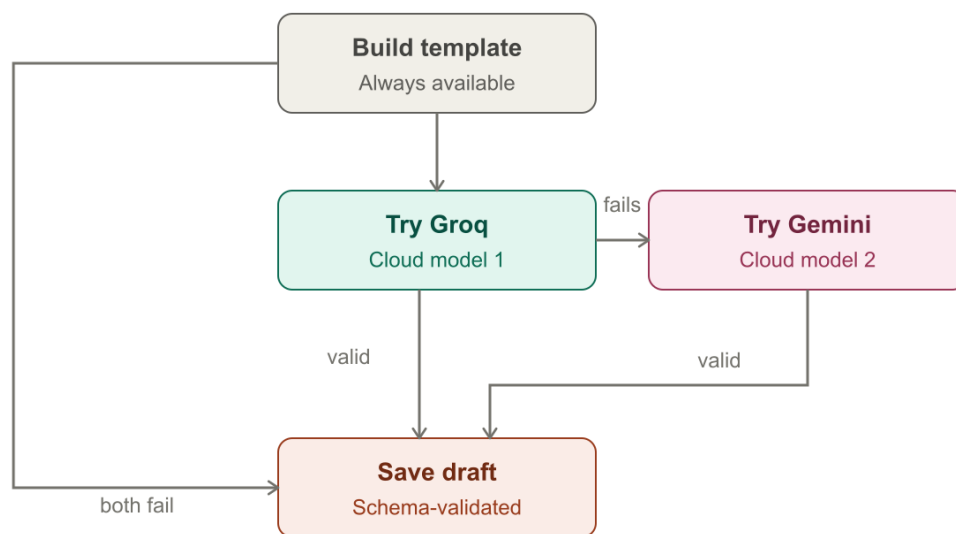


Figure 3. The escalating fallback chain: template first, then Groq, then Gemini, merging back to one saved draft either way.

Once a draft exists, it is reviewed rather than trusted outright:

- The draft is saved with status "draft", along with which adapter produced it (template, groq, or gemini) and that adapter's version.
- A doctor — specifically a doctor, not an ANM/ASHA — reviews the text and may edit it before deciding.
- Approving records the reviewer's identity and an approval timestamp alongside the existing model/version metadata.
- A patient account is only ever shown an approved summary — the query that lists summaries filters out drafts entirely for that role, not just in the UI but in the API response itself.

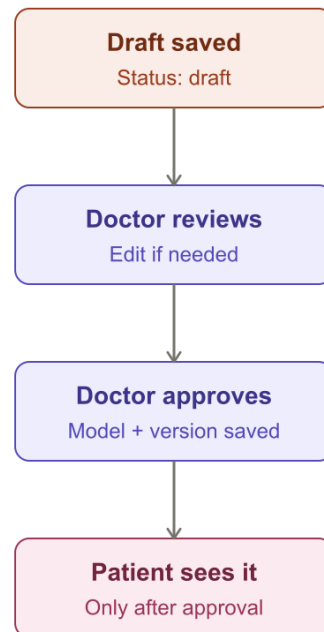


Figure 4. A draft is never patient-visible until a doctor has reviewed, optionally edited, and explicitly approved it.

6. Cross-cutting principles

- One access rule, reused everywhere: facility match or active consent, checked in a single backend function, never duplicated into database row-level security.
- Append-only clinical facts: no edit or delete route exists for an encounter anywhere in the API — continuity of care never depends on a silent overwrite.
- Realtime is a notification, not a source of truth: every Socket.IO event triggers a REST re-fetch; the event payload itself is never acted on directly.
- Idempotency by design: every offline-queued mutation carries a client-generated key, so a retried request after a dropped connection cannot create a duplicate row.
- Everything sensitive is audited: viewing a record, creating an encounter, granting or revoking consent, generating or approving an AI summary all write to an append-only audit log.
- AI is assistance, never autonomous: every model-authored summary starts as a draft, requires a doctor's approval, and carries permanent model/version attribution.

7. Guardrails

- No real patient data in this environment, ever — synthetic and demo data only.
- Secrets (Supabase service role key, Groq and Gemini API keys) live only in the backend's environment configuration and never reach a client bundle.
- No paid SMS OTP, no paid TURN infrastructure, no paid AI tier required — the entire stack runs on free tiers.
- The district dashboard shows counts and averages only — never a patient name or a clinical note.
- Voice input and multilingual UI complement the interface; every critical action remains reachable by ordinary touch input in any language.

8. Deployment

Three pieces deploy independently, because only one of them can run as a long-lived process:

Part	Host	Reason
Express API + Socket.IO	Render	Socket.IO holds persistent WebSocket connections; a serverless function cannot keep one open.
Mobile web export	Vercel	A pure static single-page build, exactly what Vercel serves best.
Admin console	A second Vercel project	Also a static Vite build, but with its own build command and output directory.